

UGV Path Planning using A* and Dynamic Replanning

By: **Prachi Lahoti**, SE24UCSE218

1. Introduction

This project focuses on path planning for an Unmanned Ground Vehicle (UGV) in both static and dynamic environments. The goal is to find the shortest path from a start node to a goal node on a grid map while avoiding obstacles.

2. A* Algorithm (Static Environment)

The A* (A-Star) algorithm is an informed search algorithm used to find the shortest path in a graph. It combines the advantages of Dijkstra's Algorithm and Greedy Best-First Search.

Evaluation Function:

$$f(n)=g(n)+h(n)$$

Where:

- $g(n)$ = cost from start to current node
- $h(n)$ = heuristic estimate (Manhattan distance)

In this project, A* is used when all obstacles are known beforehand (static environment).

3. Dynamic Path Planning

In real-world scenarios, obstacles may appear or disappear dynamically. In such cases, the UGV cannot rely on a precomputed path.

Approach used:

- Compute initial path using A*
- Move step-by-step
- Detect new obstacles
- Recompute path using A*

This process continues until the goal is reached.

4. Algorithm Workflow

1. Initialize grid and obstacle density
2. Take user input for start and goal
3. Run A* to compute initial path

4. Move the robot step-by-step
5. Introduce dynamic obstacles randomly
6. Re-run A* if path is blocked
7. Continue until goal is reached or no path exists

5. Requirements

- Python 3.x
- Libraries used:
 - `heapq` (priority queue)
 - `random` (for obstacle generation)

6. Input and Output

Inputs:

- Grid size
- Start position (row, col)
- Goal position (row, col)
- Obstacle density

Outputs:

- Step-by-step movement
- Final path
- Total steps taken
- Success/Failure message

Output Example for UGV:

Code

Enter START position:

Row (0-69): @

Col (0-69): @

Enter GOAL position:

Row (0-69): 10

Col (0-69): 10

Obstacle density:

1. Low (~10%)

2. Medium (~25%)

3. High (~40%)

Choose [1-3]: 1

Planning path from (0, 0) to (10, 10) with low obstacle density ...
The battlefield environment is represented as a 70 x 70 grid map.

Path found : 21 steps (cost = 20)

Start + Goal: (0, 0) + (10, 10)

Dynamic Example:

Code

```
Enter grid size (e.g., 20 or 70): 20
Enter START position:
Row (0-19): 4
Col (0-19): 4
Enter GOAL position:
Row (0-19): 13
Col (0-19): 13
```

```
Dynamic obstacle density:
```

```
1. Low (~1%)
2. Medium (~2%)
3. High (~5%)
Choose [1-3]: 2
```

```
Planning in dynamic environment ...
```

```
Step 1: Moving to (4, 5)
Step 2: Moving to (4, 6)
Step 3: Moving to (5, 6)
...
Step 21: Moving to (12, 11)
X No_path available!
```

7. Conclusion

The project demonstrates how A* efficiently solves static path planning problems, and how dynamic replanning enables navigation in uncertain environments. Such techniques are widely used in robotics, autonomous vehicles, and military applications.